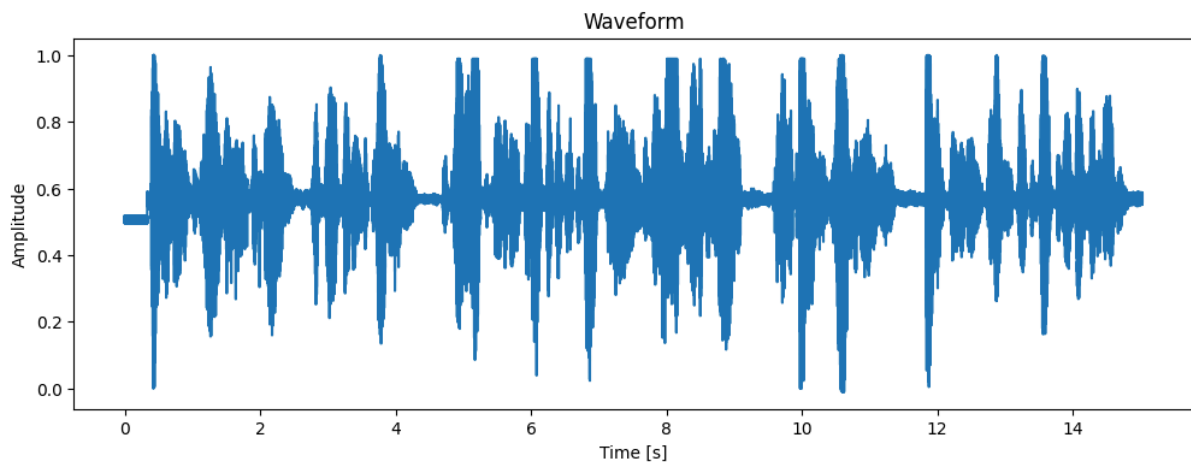


นิสิตจะทำการแบ่งขั้นตอนการทำงานตามบล็อกโค้ดในไฟล์ ipynb โดยหากจะรันโค้ดจะต้องรันตามบล็อกจากสูงไปต่ำเพื่อป้องกันการ Error ในส่วนของวิธีการทำงานของโค้ดนิสิตจะอธิบายไว้ในรูปแบบ Comment ในไฟล์ ipynb อยู่แล้ว

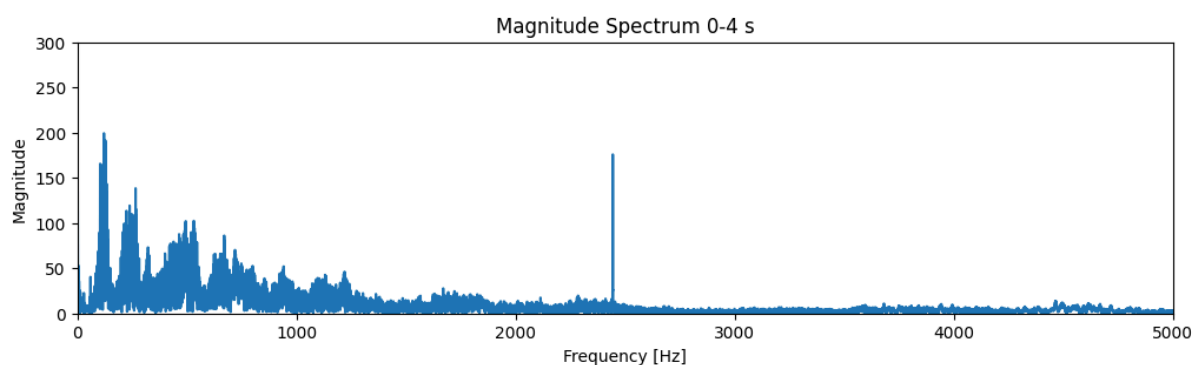
1.) อ่านไฟล์เสียงและพล็อตกราฟ Waveform



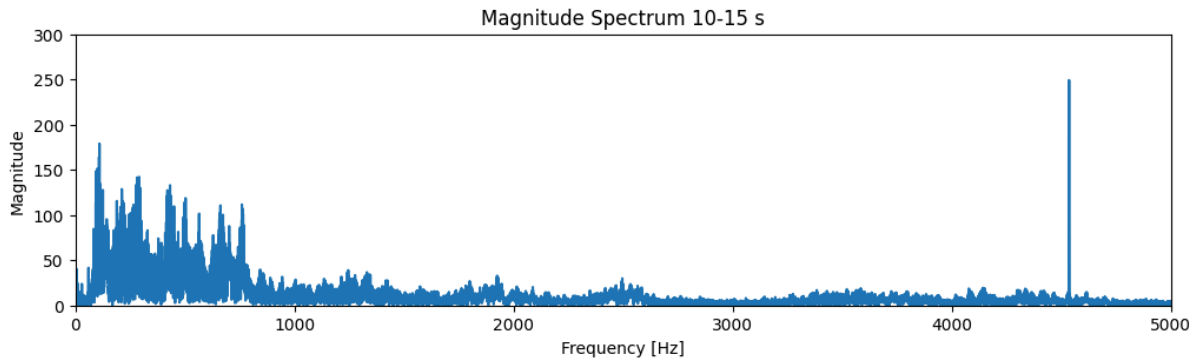
ภาพที่ 1 : Waveform ของเสียง

ในบล็อกแรกจะประกอบไปด้วยการ import lib การอ่านและแสดงข้อมูลพื้นฐานของไฟล์เสียง โดยก่อนจะพล็อตกราฟนิสิตได้ทำการ Normalize Data ให้อยู่ในช่วง 0-1 ไม่ได้เห็นความแตกต่างชัดเจนในบล็อกนี้แต่นิสิตทำไว้เพื่อเทียบกับไฟล์หลังผ่านการกรองภายหลัง

2.) แปลงและพล็อตกราฟใน Frequency Domain



ภาพที่ 2 : Waveform ของเสียงในช่วง 0-4 วินาที

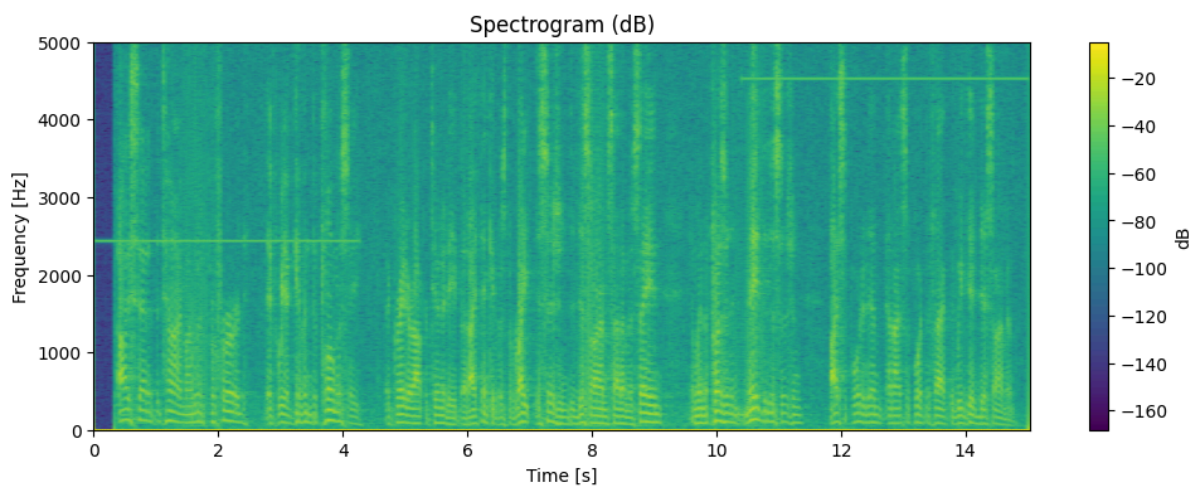


ภาพที่ 3 : Waveform ของเสียงในช่วง 10-15 วินาที

นิสิตทำการสร้างฟังก์ชันเพื่อแปลง Data ใน Time Domain ไป Freq Domain โดยใช้ FFT จากนั้นดึงแค่ค่า Real มาใช้ โดยฟังก์ชันนี้สามารถเลือกช่วงเวลาที่ต้องการให้พล็อตกราฟเองได้

ก่อนหน้านี้นิสิตได้ทำการนำไฟล์ mp3 มาเปิดฟังและได้ทราบว่าในช่วงเวลาประมาณ 0-4 วินาที และ 10-15 วินาที มีเสียงจืดหรือเสียงรบกวนปนอยู่ นิสิตจึงทำการพล็อตกราฟความถี่ในช่วงเวลานั้นออกมา พบว่ามีบางความถี่ที่มีค่ามากกว่าปกติ ($\approx 2500, 4500 \text{ Hz}$)

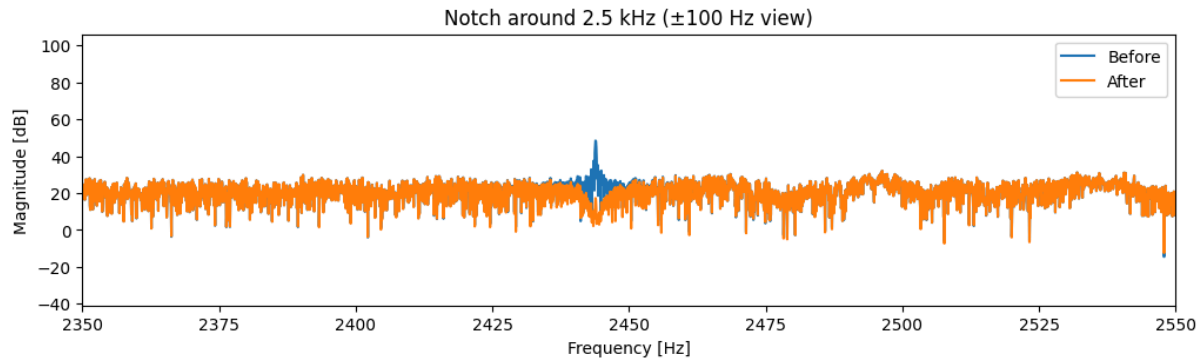
3.) สร้าง Spectrogram เพื่อยืนยันความถี่ที่ผิดปกติ



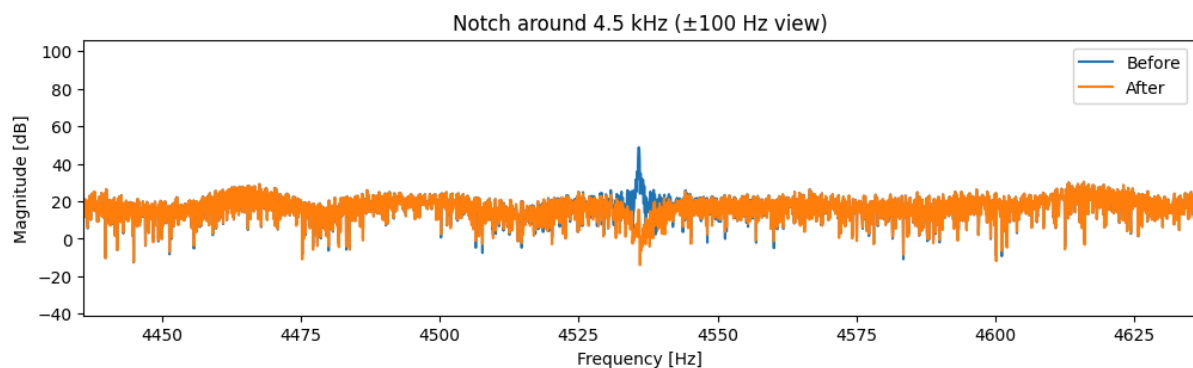
ภาพที่ 4 : Spectrogram ของย่านความถี่ 0-5000 Hz

ก่อนนำค่าไปพล็อตนิสิตได้ทำการ Scale ข้อมูลให้อยู่ในช่วง dB เพื่อกันไม่ให้ข้อมูลกระจายมากเกินไป จากนั้นนิสิตทำการพล็อตกราฟ Spectrogram เพื่อยืนยันว่าความถี่รบกวนนั้นตรงกับที่พบในบล็อก 2 หรือไม่ โดยหากสังเกตจากกราฟจะพบว่าในย่านความถี่ $\approx 2500, 4500 \text{ Hz}$ นั้นมีเส้นซึ่งแสดงถึงความถี่ที่มีความเข้มสูงผิดปกติในช่วงเวลาที่คาดเดาไว้พอดี

4.) Filter สัญญาณด้วย Notch



ภาพที่ 5 : เปรียบเทียบ Frequency Domain ช่วง 2.5 kHz ก่อน/หลัง ทำการ Filter



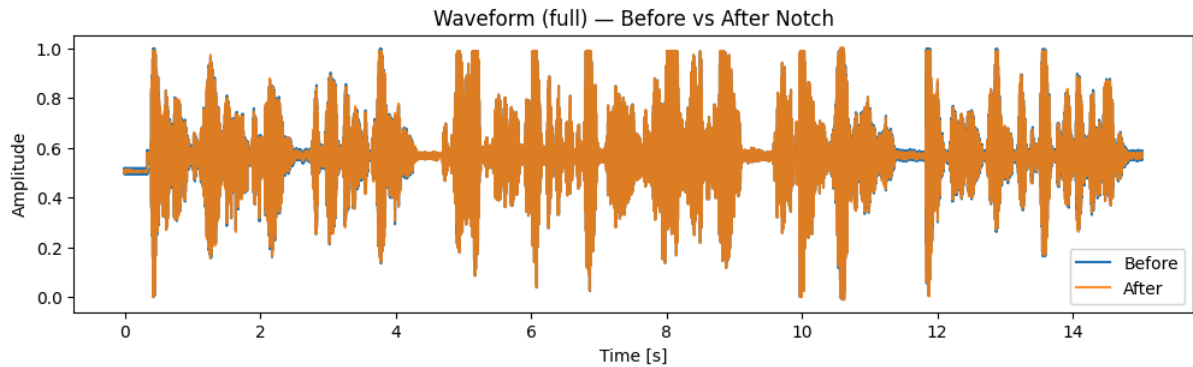
ภาพที่ 6 : เปรียบเทียบ Frequency Domain ช่วง 4.5 kHz ก่อน/หลัง ทำการ Filter

นิสิตทำการ Normalize Data ก่อนเพื่อนำไปเข้าฟังก์ชัน `notch_with_depth(y, sr, freqs, BW, att_db)` ซึ่ง Input จะมีดังนี้

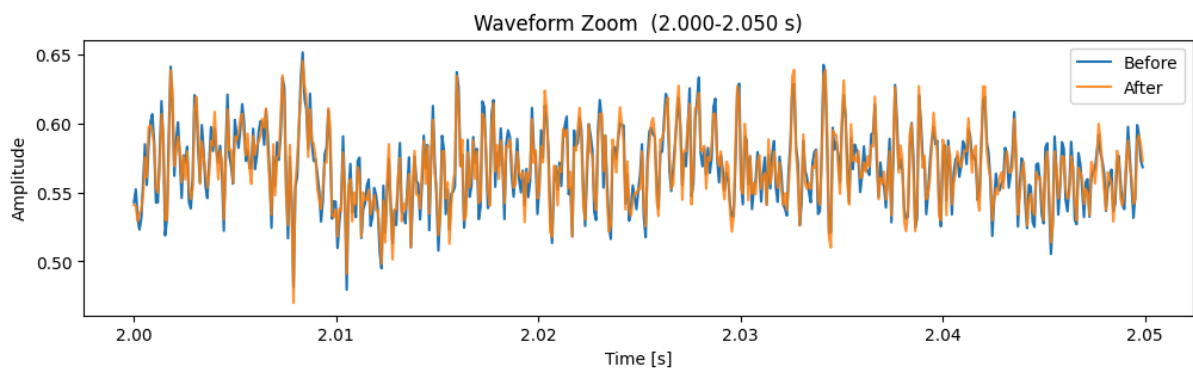
- `y` = Data ที่ถูก Normalize แล้ว
- `sr` = Sampling Rate
- `freqs` = ตำแหน่งกึ่งกลางของความถี่ที่ต้องการ Filter
- `BW` = ขนาด Bandwidth ที่ต้องการ Filter
- `att_db` = ความดังที่ต้องการลดลง (dB)

โดยนิสิตได้ทำการใส่ `freqs` ตามตำแหน่งที่พบความถี่ที่ผิดปกติ จากนั้นกำหนด Bandwidth กับ `att_db` ให้เหมาะสม ไม่มากจนส่งผลให้สัญญาณเกิดการเปลี่ยนแปลงมากเกินไป หลังจากนั้นนิสิตได้ทำการจำลองกราฟ ก่อน/หลัง ทำการ Filter ออกมาดังรูปเพื่อให้เห็นความแตกต่าง จากนั้นก็ทำการแปลงและเซฟไฟล์หลังทำการ Filter เสร็จ

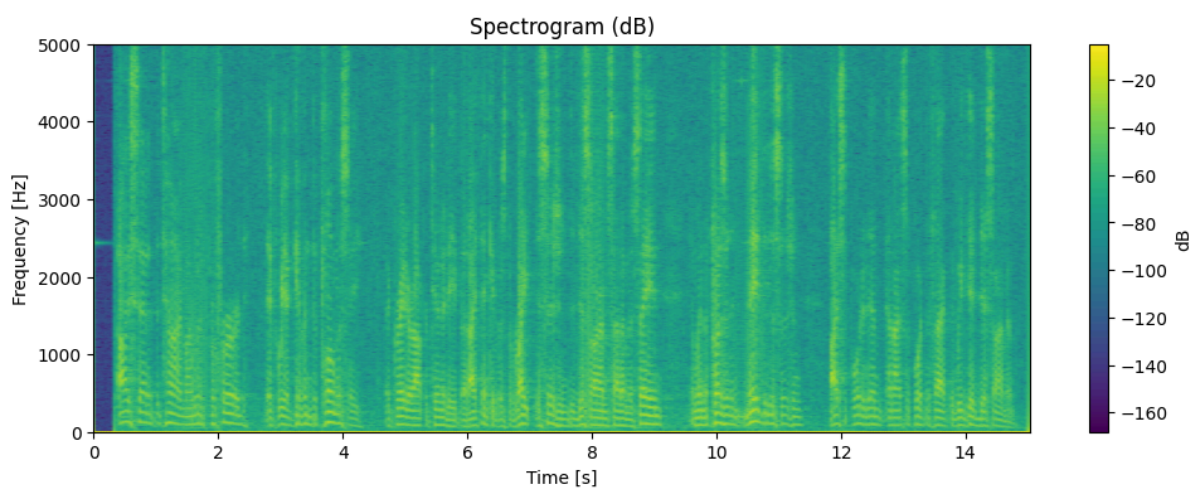
5.) สร้างกราฟหลังการ Filter เพื่อเปรียบเทียบ



ภาพที่ 7 : เปรียบเทียบ Time Domain ทุกช่วง ก่อน/หลัง ทำการ Filter



ภาพที่ 8 : เปรียบเทียบ Time Domain ช่วง 2-2.05 วินาที ก่อน/หลัง ทำการ Filter



ภาพที่ 9 : Spectrogram หลังทำการ Filter

ในภาพเปรียบเทียบ Waveform สังเกตได้ว่า Amplitude นั้นแตกต่างกันเพียงนิดเดียว หลังจาก Filter ความถี่บางช่วงออก จะเห็นได้ชัดกว่าในภาพที่ซูมเข้า และในภาพต่อไปแสดงกราฟ

Spectrogram สังเกตได้ว่าเส้นสว่างๆที่เคยเห็นได้ชัดในบางความถี่นั้นกลมกลืนไปกับความถี่อื่นๆ แล้ว แสดงว่าการทำ Filter ในข้อ 4 นั้นได้ผล

6.) คำนวณ MAE

```
1 def mae(y_true, y_filt):
2     n = min(len(y_true), len(y_filt))
3     a = y_true[:n].astype(np.float64)    #ปรับให้แน่ใจว่าเป็น Dtype เดียวกัน
4     b = y_filt[:n].astype(np.float64)
5     return float(np.mean(np.abs(a - b)))
6
7 mae_value = mae(y, y_notched)
8 mae_norm_pct = mae_value * 100.0    # % full-scale
9 print("MAE:", mae_value)
10 print("MAE (Normalize):", mae_norm_pct, "%")
```

MAE: 0.004296398230255875
MAE (Normalize): 0.4296398230255875 %

ภาพที่ 10 : สูตร MAE

ในขั้นตอนสุดท้ายจะทำการคำนวณความแตกต่างของ Data โดยใช้สูตร MAE ตามนี้ค่าที่ได้ออกมาจะเป็นค่าที่ Normalize อยู่แล้ว

7.) สรุปผลการทดลอง

จากการทดลองนี้ถือว่าประสบความสำเร็จเนื่องจากทำฟลิเสียงหลังทำการทดลองนั้นไม่มีเสียงรบกวนเหลือแล้ว โดยในการทดลองนี้ใช้ส่วนขยายของ Python ที่ชื่อว่า Notch ซึ่งทำหน้าที่คล้าย Bandstop เป็นตัวกรองเสียงรบกวน

โดยการที่จะหาเสียงรบกวนได้ว่าอยู่ในช่วงเวลาไหน หรือความถี่ไหนก็ต้องอาศัยการทดสอบด้วยตัวเองไม่ว่าจะเป็น การเปิดฟังเสียง การสร้างกราฟเวลา หรือการสร้างกราฟความถี่ โดยท้ายที่สุดแล้วนิสิตก็ได้พบว่าเสียงรบกวนนั้นอยู่ในช่วงไหน และความถี่เท่าไรบ้างจากนั้นจึงทำการกรองเสียงรบกวน

หลังจากทำการทำการกรองเสียงรบกวนเรียบร้อยแล้ว เพื่อความถูกต้องนิสิตต้องวนกลับไปทำขั้นตอนเดิมๆเช่น ฟังเสียง สร้างกราฟเวลา และกราฟความถี่ เพื่อตรวจสอบความแตกต่างหลังกรองความถี่เรียบร้อยแล้ว และสุดท้ายก็คำนวณหาค่า MAE ซึ่งผลก็ออกมาเป็นที่น่าพึงพอใจ